Institut Africain d'Informatique
Représentation du Togo

Préparation de l'examen
Langage C — 2026

Chargé de cours : M. AKOMAGBE Innocent

Exercice 1 : Nombres complexes

On souhaite représenter un nombre complexe $z \in \mathbb{C}$ par deux réels re (la partie réelle) et img (la partie imaginaire) : $z = \boldsymbol{re} + \boldsymbol{img} \cdot i$

1. Donner la déclaration d'un type nombre complexe nommé `Complexe` représentant z.
2. Implémentez les fonctions suivantes :

Lire_re — double

```
double Lire_re(Complexe z)
```
Renvoie la partie réelle d'un complexe.

Exemple

```
1. Complexe z = {5.2, -3.7};
2. double re = Lire_re(z);
3. printf("%lf", re);  // affiche 5.200000
```

Lire_img — double

```
double Lire_img(Complexe z)
```
Renvoie la partie imaginaire d'un complexe.

Exemple

```
1. Complexe z = {5.2, -3.7};
2. double img = Lire_img(z);
3. printf("%lf", img);  // affiche -3.700000
```

Ecrire_re — Complexe

```
Complexe Ecrire_re(double re, Complexe z)
```
Affecte la partie réelle d'un complexe avec la valeur spécifiée en argument.

Exemple

```
1. Complexe z = {5.2, -3.7};
2. z = Ecrire_re(10.5, z);
3. printf("%lf %lf", z.re, z.img);  // affiche 10.500000 -3.700000i
```

Ecrire_img — Complexe

```
Complexe Ecrire_img(double img, Complexe z)
```
Affecte la partie imaginaire d'un complexe avec la valeur spécifiée en argument.

Exemple

```
1. Complexe z = {5.2, -3.7};
2. z = Ecrire_img(8.4, z);
3. printf("%lf %lf", z.re, z.img);  // affiche 5.200000 8.400000i
```

Representation_z — void

```
void Representation_z(Complexe z)
```
Affiche la représentation d'un complexe z.

Exemple

```
1. Complexe z = {3.0, 4.0};
2. Representation_z(z);  // affiche par exemple "3.00 + 4.00i"
```

Conjugue

Complexe

```
Complexe Conjugue(Complexe z)
Retourne le conjugué du nombre complexe z.
Rappel : si z = re + img·i, alors z̄ = re − img·i
```

Exemple

```
1. Complexe z = {3.0, 4.0};
2. Complexe zc = Conjugue(z);
3. Representation_z(zc); // affiche "3.00 - 4.00i"
```

Argument

double

```
double Argument(Complexe z)
Calcule et retourne l'argument (en radians) du nombre complexe z.
Rappel : θ = atan2(img, re).
Retourne 0.0 si z est le complexe nul.
```

Exemple

```
1. Complexe z = {1.0, 1.0};
2. double arg = Argument(z);
3. printf("%lf", arg); // affiche 0.785398 (π/4)
```

Puissance

Complexe

```
Complexe Puissance(Complexe z, int n)
Calcule et retourne z^n par multiplications successives.
Si n est nul, retourne le complexe {1.0, 0.0}.
```

Exemple

```
1. Complexe z = {1.0, 1.0};
2. Complexe r = Puissance(z, 3);
3. Representation_z(r); // affiche "-2.00 + 2.00i"
```

3. Pour chacune des opérations suivantes écrire la fonction ou la procédure :
   - 3.1. L'opération calculant le module d'un nombre complexe $|z| = \sqrt{re^2 + img^2}$
   - 3.2. L'opération testant l'égalité de deux nombres complexes.
   - 3.3. L'opération calculant la somme de deux nombres complexes. Rappel : $z_1 + z_2 = (re_1 + re_2) + i(img_1 + img_2)$
   - 3.4. L'opération calculant le produit de deux nombres complexes.

   Rappel : $z_1 z_2 = (re_1 re_2 - img_1 img_2) + i(re_1 img_2 + re_2 img_1)$
   - 3.5. L'opération donnant l'inverse d'un nombre complexe.

   rappel : si z est le complexe nul, cela doit donner une erreur, sinon $z^{-1} = \frac{\overline{z}}{|z|^2}$
   - 3.6. L'opération calculant le rapport de deux nombres complexes (qui doit donner une erreur si le dénominateur est nul).

   rappel : $\frac{z_1}{z_2} = z_1 \frac{1}{z_2}$
   - 3.7. Soit tz un tableau de N nombres complexes ($N \leq 100$). En utilisant les actions paramétrées précédentes, écrire un algorithme qui :
     - 3.7.a. Affichez toutes les valeurs de tz.
     - 3.7.b. Permet de saisir et de remplir le tableau tz avec des nombres complexes. On pourra générer les deux nombres formant les nombres complexes.
     - 3.7.c. Affiche l'élément de tz ayant le plus grand module. Puis vérifie l'existence de son conjugué dans tz.
     - 3.7.d. Calcule la somme zsom et le produit zprod des éléments non nuls du tableau Tz.

Exercice 2 : Gestion de notes

On souhaite informatiser la gestion des notes des étudiants de l'IAI-Togo en langage c.

Un étudiant sera identifié par :

— son matricule;

— son nom;

— son prénom;

— sa date de naissance (qui sera décomposée en jour, mois, et année) ;

— un tableau contenant les moyennes aux 11 UEs;

1. Donnez les structures de données nécessaires à la définition d'un étudiant et d'une structure permettant la gestion d'une promotion de 700 étudiants.
2. Implémentez les fonctions suivantes :

SaisirEtudiant — Etudiant

```
Etudiant SaisirEtudiant()
Permet de saisir les informations d'un étudiant.
```

AfficherEtudiant — void

```
void AfficherEtudiant(Etudiant e)
Affiche toutes les informations d'un étudiant : identité, date de naissance et les 11 moyennes.
```

CalculerMoyenneGenerale — float

```
float CalculerMoyenneGenerale(Etudiant e)
Calcule et retourne la moyenne générale d'un étudiant sur les 11 UEs (moyenne arithmétique).
```

AfficherPromotion — void

```
void AfficherPromotion(Etudiant promotion[])
Affiche la liste de tous les étudiants de la promotion avec leur moyenne générale.
```

RechercherParNumero — int

```
int RechercherParNumero(Etudiant promotion[], int num)
Recherche un étudiant par son numéro. Retourne son indice dans le tableau ou -1 si non trouvé.
```

RechercherParNom — int

```
int RechercherParNom(Etudiant promotion[], char nom[])
Recherche le premier étudiant dont le nom correspond (insensible à la casse). Retourne l'indice ou -1.
```

StatistiquesPromotion — void

```
void StatistiquesPromotion(Etudiant promotion[])
Calcule et affiche les statistiques de la promotion : moyenne générale de la promo, note minimale, note maximale, nombre d'étudiants avec moyenne >= 10. On pourra découper ces statistiques en fonctions.
```

TauxReussite — float

```
float TauxReussite(Etudiant promotion[])
Calcule et retourne le taux de réussite de la promotion en pourcentage :
```

$$\text{taux} = \frac{\text{nb étudiants avec moyenne} \geq 10}{\text{effectif total}} \times 100$$

Exemple

```c
float taux = TauxReussite(promotion);
printf("Taux de reussite : %.2f%%", taux); // ex: 72.43%
```

PlusJeune int

```
int PlusJeune(Etudiant promotion[])
Retourne l'indice dans le tableau de l'étudiant le plus jeune (date de naissance la plus récente).
```

Exemple

```
1. int idx = PlusJeune(promotion);
2. AfficherEtudiant(promotion[idx]);
```

PlusVieux int

```
int PlusVieux(Etudiant promotion[])
Retourne l'indice dans le tableau de l'étudiant le plus vieux (date de naissance la plus ancienne).
```

Exemple

```
1. int idx = PlusVieux(promotion);
2. AfficherEtudiant(promotion[idx]);
```

3. Dans votre fonction main, écrire les instructions pour réaliser les actions suivantes :
   3.1. Permettez à l'utilisateur de saisir les informations des 700 étudiants;
   3.2. Affichez la liste brute des étudiants;
   3.3. Affichez la moyenne générale de tous les étudiants;
   3.4. Recherchez un étudiant dont le numéro est demandé à l'utilisateur et affichez ses informations s'il est trouvé;
   3.5. Recherchez un étudiant par son nom (nom saisi par l'utilisateur) et affichez le résultat;
   3.6. Affichez les statistiques de la promotion.

Exercice 3 : Gestion des employés

Considérons les structures suivantes :
Une Date est définie par les champs :
— jour ;
— mois ;
— année.

Une Adresse est définie par les champs :
— Numéro ;
— Rue ;
— Commune ;
— CodePostal.

Un Employé est défini par les champs :
— Nom ;
— Prénom ;
— Résidence ;
— DateDeNaissance.

1. Définir la structure Employé.
2. Définir un tableau appelé tabEmployes de 100 éléments de type Employé.
3. Implémentez les fonctions suivantes :

SaisirEmploye — Employe

```
Employe SaisirEmploye(void)
```

Saisit les informations d'un employé.

AfficherEmploye — void

```
void AfficherEmploye(Employe e)
```

Affiche les informations d'un employé (nom, prénom, date de naissance, commune, etc.).

ChercherEmployeNom — int

```
int ChercherEmployeNom(Employe tabEmployes[], int nb, char nom[])
```

Cherche un employé par son nom dans le tableau. Retourne l'indice de l'employé s'il est trouvé, sinon -1.

CompterParCommune — int

```
int CompterParCommune(Employe tabEmployes[], int nb, char commune[])
```

Compte le nombre d'employés résidant dans la commune spécifiée.

CalculerAge — int

```
int CalculerAge(Date naissance, int anneeActuelle)
```

Calcule l'âge d'une personne à partir de sa date de naissance et de l'année actuelle.

AgeEmploye — int

```
int AgeEmploye(Employe e, int anneeActuelle)
```

Calcule l'âge d'un employé.

CompterMajeurs — int

```
int CompterMajeurs(Employe tabEmployes[], int nb, int anneeActuelle, int ageSeuil)
```

Compte les employés majeurs selon le seuil d'âge donné.

AgeMoyen — float

```
float AgeMoyen(Employe tabEmployes[], int nb, int anneeActuelle)
```

Calcule l'âge moyen des employés dans le tableau.

MemeCommune — int

```
int MemeCommune(Employe e1, Employe e2)
```

Retourne 1 si les deux employés habitent la même commune, 0 sinon.

NeeAvant — int

```
int NeeAvant(Employe e1, Employe e2)
```

Retourne 1 si l'employé e1 est né avant l'employé e2, -1 sinon.

DateValide — int

```
int DateValide(Date d)
```

Vérifie qu'une date est valide (jour, mois, année cohérents).
Retourne 1 si la date est valide, 0 sinon.
Exemples de dates invalides : 30 février, mois 13, jour 0.

Exemple

```
1. Date d = {30, 2, 2000};
2. printf("%d", DateValide(d)); // affiche 0 (invalide)
3. Date d2 = {15, 6, 1995};
4. printf("%d", DateValide(d2)); // affiche 1 (valide)
```

PlusAge

int

```
int PlusAge(Employe tabEmployes[], int nb, int anneeActuelle)
```

Retourne l'indice de l'employé le plus âgé dans le tableau.

Exemple

```
1. int idx = PlusAge(tabEmployes, 100, 2026);
2. AfficherEmploye(tabEmployes[idx]);
```

RechercheAvancee

void

```
void RechercheAvancee(Employe tabEmployes[], int nb, char commune[], int ageMin, int ageMax, int anneeActuelle)
```

Affiche les noms et prénoms des employés résidant dans `commune` et dont l'âge est compris entre `ageMin` et `ageMax` inclus.

Exemple

```
1. RechercheAvancee(tabEmployes, 100, "Lomé", 25, 40, 2026);
2. // Affiche les employés de Lomé âgés de 25 à 40 ans
```

4. Remplir le tableau tabEmployes avec des saisies d'employés.
5. Afficher les noms et prénoms des employés résidant dans une commune qui sera donnée par l'utilisateur.
6. Afficher les noms et prénoms des employés résidant dans une commune et nés avant une date donnée. Le nom de la commune et la date seront donnés par l'utilisateur.
7. Demander à l'utilisateur un numéro de mois, puis afficher les employés dont l'anniversaire tombe dans ce mois.
8. Proposer des cas d'usage pour les autres fonctions non couvertes.